IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Pankaj Bisht
21st March 2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- **Summary of methodologies**
  - Space X data collection:
    - Space X API
    - Web Scraping
  - Data Wrangling
  - Exploratory data analysis (EDA):
    - Using SQL
    - Visualization using Pandas, Matplotlib and Seaborn
    - Feature Engineering using Pandas
  - Interactive launch site data analysis:
    - Using Folium
    - Interactive dashboard using Plotly Dash
  - Machine Learning predictive analysis for landing:
    - Support Vector Machines (SVM)
    - Classification Trees
    - Logistic Regression

- **Summary of all results**
  - EDA results
  - Interactive launch site analysis
  - Interactive launch site dashboard
  - Predictive landing analysis

Introduction

- **Project background**
  - The commercial space age is highlighted with companies like Virgin Galactic, Rocket Lab, Blue Origin, and SpaceX leading the way.
  - SpaceX's achievements include sending spacecraft to the International Space Station and launching manned missions, primarily due to cost-effective rocket launches.
  - Space X Falcon 9 rocket launches costs 62 million dollars while other providers cost upward of 165 million dollars.
  - Much of the savings is because Space X can reuse the first stage.
- **Problem that we are trying to solve**
  - If we can determine if the first stage of Falcon 9 will land, we can determine the cost of a launch.
  - This information can be used by an alternate company to bid against SpaceX for a rocket launch.

Section 1

Methodology

Data Collection

- **SpaceX Falcon 9 Data Collection –**
  - First process of data collection was using **SpaceX API** 'GET request'.
  - Then define a series of helper functions to extract information using identification numbers in the launch data using API.
  - Decode the response content as a Json and turn it into a Pandas dataframe.
  - Second process of data collection was using **Web Scraping** Falcon 9 and Falcon Heavy Launches records from Wikipedia. https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches updated on 9th June 2021.
  - Data was extracted from webpage HTML table using Beautiful Soup and some helper functions.
  - Finally, the data was turned into a Pandas dataframe.

Data Collection – SpaceX API

- Request and parse the SpaceX launch data from it's API using the HTTP GET request.
- Decode the response content as a Json using .json()
- Turn it into a Pandas dataframe using .json_normalize()
- Use API to get information about the launches using the IDs given for each launch – "rocket, payloads, launchpad, and cores".
- Store data from these requests in lists and create a new dataframe.
- Use helper functions to fill above variables.
- Construct Pandas dataset.
- Finally filter the dataframe to only include Falcon 9 launches.
- GitHub URL of SpaceX API calls notebook- https://github.com/pankajbisht11-design/SpaceY/blob/f9de3f5b53dbeb21a2c893b3fac3931798dc5999/1%20Collecting%20the%20data.ipynb

Now let's start requesting rocket launch data from SpaceX API with the following URL:

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
response = requests.get(spacex_url)
```

To make the requested JSON results more consistent, we will use the following static response object for this project:

```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'
```

We should see that the request was successfull with the 200 status response code

```
response=requests.get(static_json_url)
```

```
response.status_code
```

```
200
```

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
# Use json_normalize meethod to convert the json result into a dataframe
respjson = response.json()
data = pd.json_normalize(respjson)
```

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. called `data_falcon9`.

```
# Hint data['BoosterVersion']!='Falcon 1'
filt =  df['BoosterVersion']!=  'Falcon 1'
data_falcon9 = df.loc[filt]
data_falcon9.head()
```

Data Collection - Scraping

- Perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.
- Create a BeautifulSoup object from the HTML response.
- Find the third table on the wiki page that contains the actual launch records.
- Iterate through the <th> elements and apply the provided extract_column_from_header() to extract column name.
- Create a data frame by parsing the launch HTML tables.
- After filling the parsed launch record values into launch_dict and then create a Pandas dataframe.
- GitHub URL of web scraping notebook- https://github.com/pankajbisht11-design/SpaceY/blob/886ce018c6acda0bfdfd4d3c625703278d67860a/2%20Web%20scraping%20Falcon%209%20and%20Falcon%20Heavy%20Launches%20Records%20from%20Wikipedia.ipynb

First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.

```
# use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers = headers)
```

Create a `BeautifulSoup` object from the HTML `response`

```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content

soup = BeautifulSoup(response.content, 'html.parser')
```

```
# Use the find_all function in the BeautifulSoup object, with element type `table`
# Assign the result to a list called `html_tables`
html_tables = soup.find_all('table')
```

Starting from the third table is our target table contains the actual launch records.

```
# Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)
```

We will create an empty dictionary with keys from the extracted column names in the previous task. Later, this dictionary will be converted into a Pandas dataframe

```
launch_dict= dict.fromkeys(column_names)

# Remove an irrelvant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]
```

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

```
df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

Data Wrangling

- Load Space X dataset, Identify and calculate the percentage of the missing values in each attribute.
- Use the method value_counts() on the column LaunchSite to determine the number of launches on each site.
- Use the method .value_counts() to determine the number and occurrence of each orbit in the column Orbit.
- Use the method .value_counts() on the column Outcome to determine the number of landing_outcomes.
- Using the Outcome, create a list where the element is zero if the corresponding row in Outcome is in the set bad_outcome; otherwise, it's one. Then assign it to the variable landing_class.
- This variable will represent the classification variable that represents the outcome of each launch.

- GitHub URL of data wrangling-https://github.com/pankajbisht11-design/SpaceY/blob/886ce018c6acda0bfdfd4d3c625703278d67860a/3%20Data%20wrangling.ipynb

Identify and calculate the percentage of the missing values in each attribute

```
df.isnull().sum()/len(df)*100
```

Use the method `value_counts()` on the column `LaunchSite` to determine the number of launches on each site:

```
# Apply value_counts() on column LaunchSite
launches = df['LaunchSite'].value_counts()
launches
```

```
# Apply value_counts on Orbit column

occurence = df['Orbit'].value_counts()
occurence
```

Use the method `.value_counts()` on the column `Outcome` to determine the number of `landing_outcomes`

```
# landing_outcomes = values on Outcome column
landing_outcomes = df['Outcome'].value_counts()
landing_outcomes
```

```
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise

df['Class'] = df['Outcome'].apply(lambda x: 0 if x in bad_outcomes else 1)
df['Class'].value_counts()
```

This variable will represent the classification variable that represents the outcome of each launch.

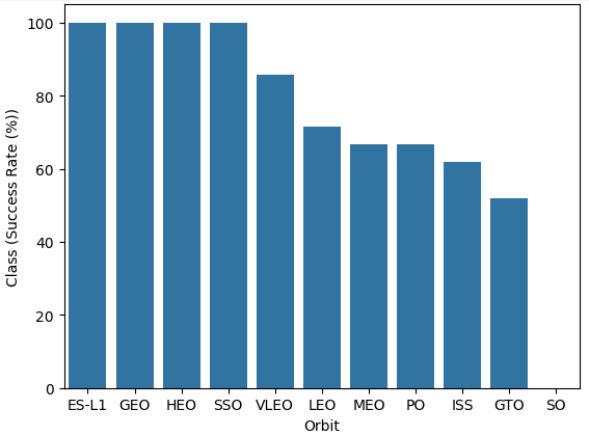
```
landing_class = df['Class']
df[['Class']].head(8)
```

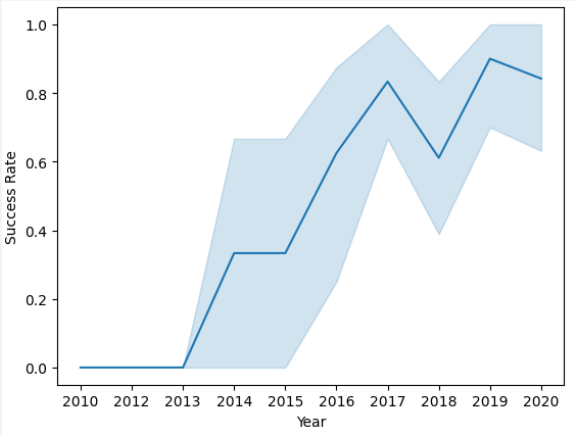
```
df.to_csv("dataset_part_2.csv", index=False)
```

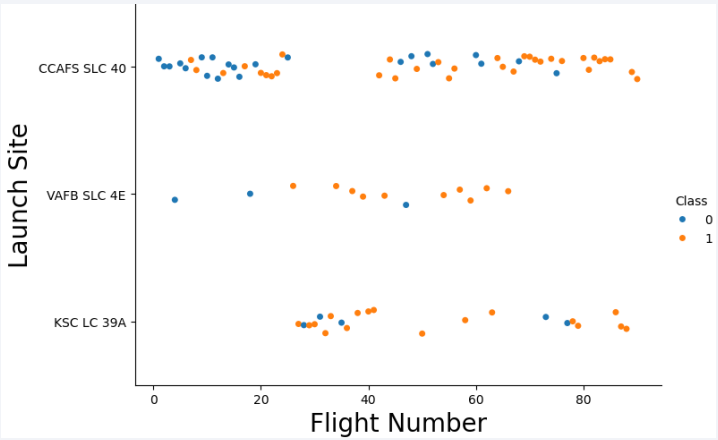
EDA with Data Visualization

EDA was performed using below plots-

- Bar plot to visualize the relationship between success rate of each orbit type.
- Line plot to visualize the launch success yearly trend.
- Scatter plots to visualize the relationship between Flight Number and Launch Site, Payload and Launch Site, FlightNumber and Orbit type, Payload and Orbit type.







- GitHub URL of EDA with data visualization notebook-https://github.com/pankajbisht11-design/SpaceY/blob/886ce018c6acda0bfdfd4d3c625703278d67860a/5%20Assignment%20Exploring%20and%20Preparing%20Data.ipynb

EDA with SQL

Following SQL queries were performed-

- Display the names of the unique launch sites in the space mission.

```
%sql SELECT DISTINCT LAUNCH_SITE as "Launch_Sites" FROM SPACEXTBL;
```

- Display 5 records where launch sites begin with the string 'CCA'.

```
%sql SELECT * FROM 'SPACEXTBL' WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

- Display average payload mass carried by booster version F9 v1.1.

```
%sql SELECT AVG(PAYLOAD_MASS__KG_) as "Payload Mass Kgs", Customer, Booster_Version FROM 'SPACEXTBL' WHERE Booster_Version LIKE 'F9 v1.1%';
```

- List the date when the first successful landing outcome in ground pad was achieved.

```
%sql SELECT MIN(DATE) FROM 'SPACEXTBL' WHERE "Landing_Outcome" = "Success (ground pad)";
```

- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000.

```
%sql SELECT DISTINCT Booster_Version, Payload FROM SPACEXTBL WHERE "Landing_Outcome" = "Success (drone ship)" AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000;
```

EDA with SQL

- List the total number of successful and failure mission outcomes.

```
%sql SELECT "Mission_Outcome", COUNT("Mission_Outcome") as Total FROM SPACEXTBL GROUP BY "Mission_Outcome";
```

- List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%sql SELECT "Booster_Version",Payload, "PAYLOAD_MASS__KG_" FROM SPACEXTBL WHERE "PAYLOAD_MASS__KG_" = (SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTBL);
```

- List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

```
%sql SELECT substr(Date,0,5), substr(Date, 6, 2),"Booster_Version", "Launch_Site", Payload, "PAYLOAD_MASS__KG_", "Mission_Outcome", "Landing_Outcome" FROM SPACEXTBL WHERE substr(Date,0,5)='2015' AND "Landing_Outcome" = 'Failure (drone ship)';
```

- Rank the count of landing outcomes between the date 2010-06-04 and 2017-03-20, in descending order.

```
%sql SELECT * FROM SPACEXTBL WHERE "Landing_Outcome" LIKE 'Success%' AND (Date BETWEEN '2010-06-04' AND '2017-03-20') ORDER BY Date DESC;
```

- GitHub URL of EDA with SQL notebook- https://github.com/pankajbisht11-design/SpaceY/blob/886ce018c6acda0bfdfd4d3c625703278d67860a/4%20Assignment%20SQL%20Notebook%20for%20Peer%20Assignment.ipynb

Build an Interactive Map with Folium

Interactive Folium map created to-

- Mark all launch sites:
  - Used folium.Circle to add a highlighted circle area with a text label on a specific coordinate.
  - Create and add folium.Circle and folium.Marker for each launch site on the site map.

- Mark the success/failed launches for each site:
  - Created MarkerCluster object for all launch records.

- Calculate the distances between a launch site to its proximities:
  - Added a MousePosition on the map to get coordinate for a mouse over a point on the map.
  - Mark down a point on the closest coastline using MousePosition and calculated the distance between the coastline point and the launch site.

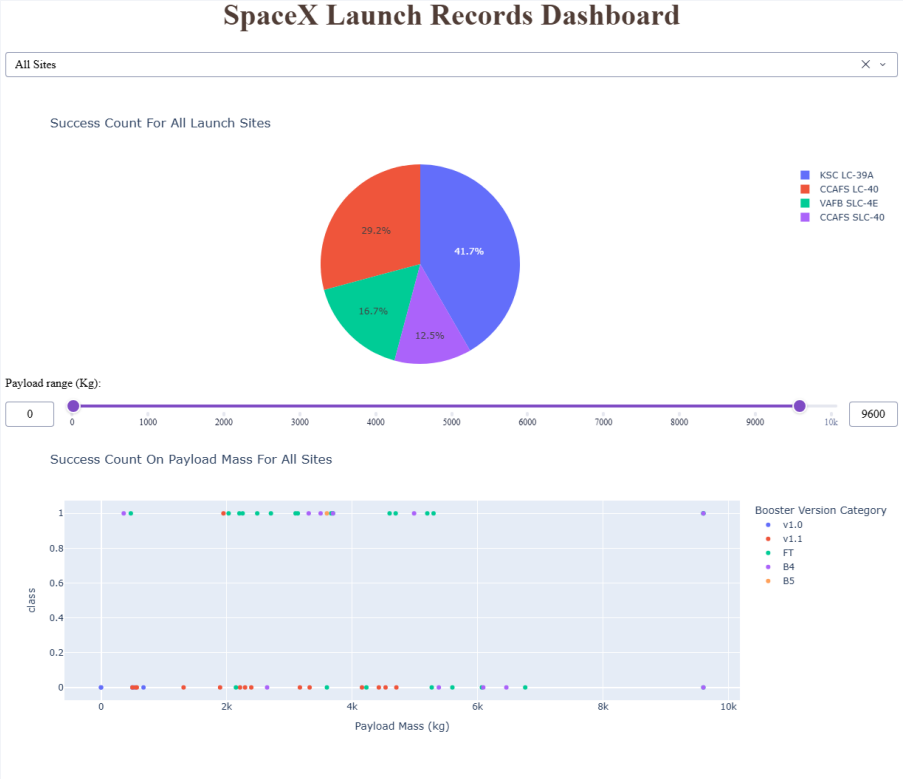
- GitHub URL of interactive map with Folium map- https://github.com/pankajbisht11-design/SpaceY/blob/e0dbe83813d01b0080915d4945235ff8788a5fc7/6%20Interactive%20Visual%20Analytics%20with%20Folium.ipynb

Build a Dashboard with Plotly Dash

Interactive dashboard created with-

- A Launch Site Drop-down Input Component.
- A callback function to render success-pie-chart based on selected site dropdown.
- A Range Slider to Select Payload.
- A callback function to render the success-payload-scatter-chart scatter plot.
- Pie charts showing the total launches by a certain sites
- Scatter graph showing the relationship with Outcome and Payload Mass (Kg) for the different booster version.

- GitHub URL of Plotly Dash lab- https://github.com/pankajbisht11-design/SpaceY/blob/e0dbe83813d01b0080915d4945235ff8788a5fc7/7%20Interactive%20Dashboard%20spacex-dash-app.py



Predictive Analysis (Classification)

For predictive analysis below steps were performed-

- Load the dataset through url.
- Create a NumPy array from the column Class in data, by applying the method to_numpy().
- Standardize the data in X using preprocessing.StandardScaler().
- Using the function train_test_split() to split the data X and Y into training and test data (test_size to 0.2 and random_state to 2).
- Further the training data is divided into validation data.
- The models are trained and hyperparameters are selected using the function GridSearchCV -
    - Create a model object then create a GridSearchCV() object.
    - Fit the object to find the best parameters from the dictionary parameters().
    - Calculate the accuracy on the test data using the method .score().
    - Plot the confusion matrix.
    - Above process is used to train models- Logistic Regression, SVM, Decision Tree Classifier, KNN
- Comparing the test data accuracy of above models, the best method was finalized.
- GitHub URL of predictive analysis lab- https://github.com/pankajbisht11-design/SpaceY/blob/bdece5e4a4dbaacb592abb15bc42974f4839595e/8%20SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

Results

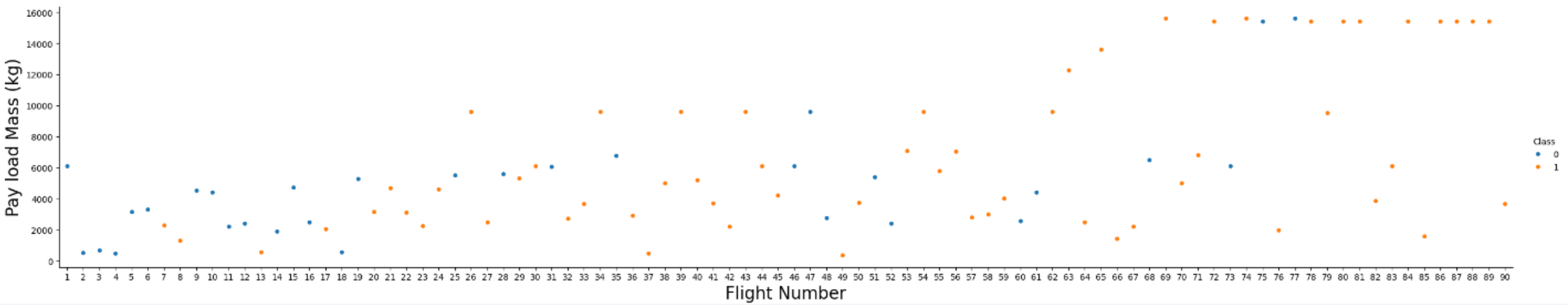
- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

Insights drawn from EDA

Flight Number vs. Payload

Flight Number vs Payload Mass

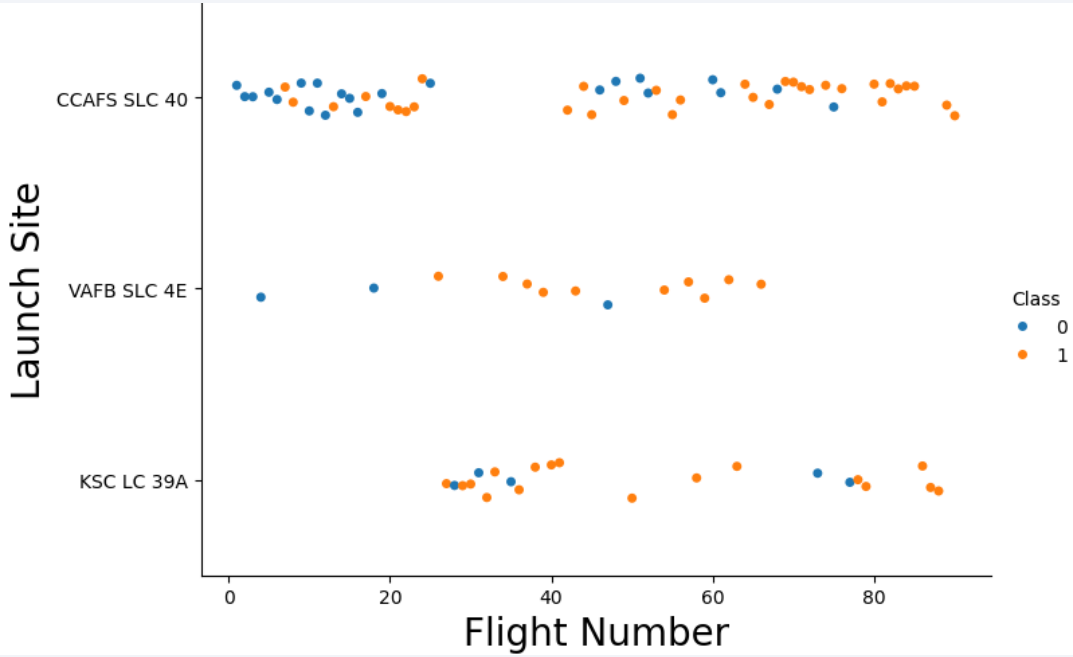


Observation-

- As the flight number increases, the first stage is more likely to land successfully.
- With more massive payloads > 9000 kg, the first stage often returns successfully.

Flight Number vs. Launch Site

Flight Number vs Launch Site

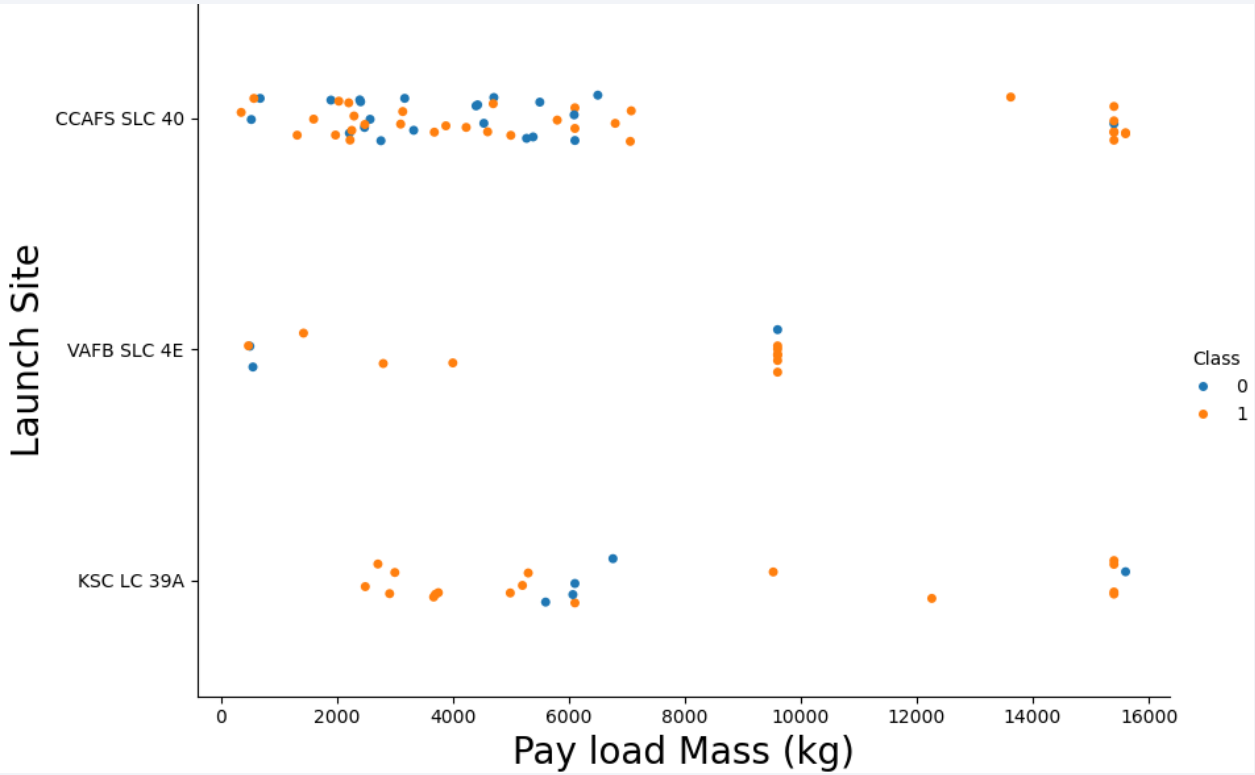


Observation-

- From the above graph, it looks like as the flight number increases in each of the 3 launch sites the success rate also increased.
- After flight no. 50, the success rate for the VAFB SLC 4E launch site is 100%
- After flight no. 80, both KSC LC 39A and CCAFS SLC 40 have a 100% success rates.

Payload vs. Launch Site

Pay Load Mass vs Launch Site

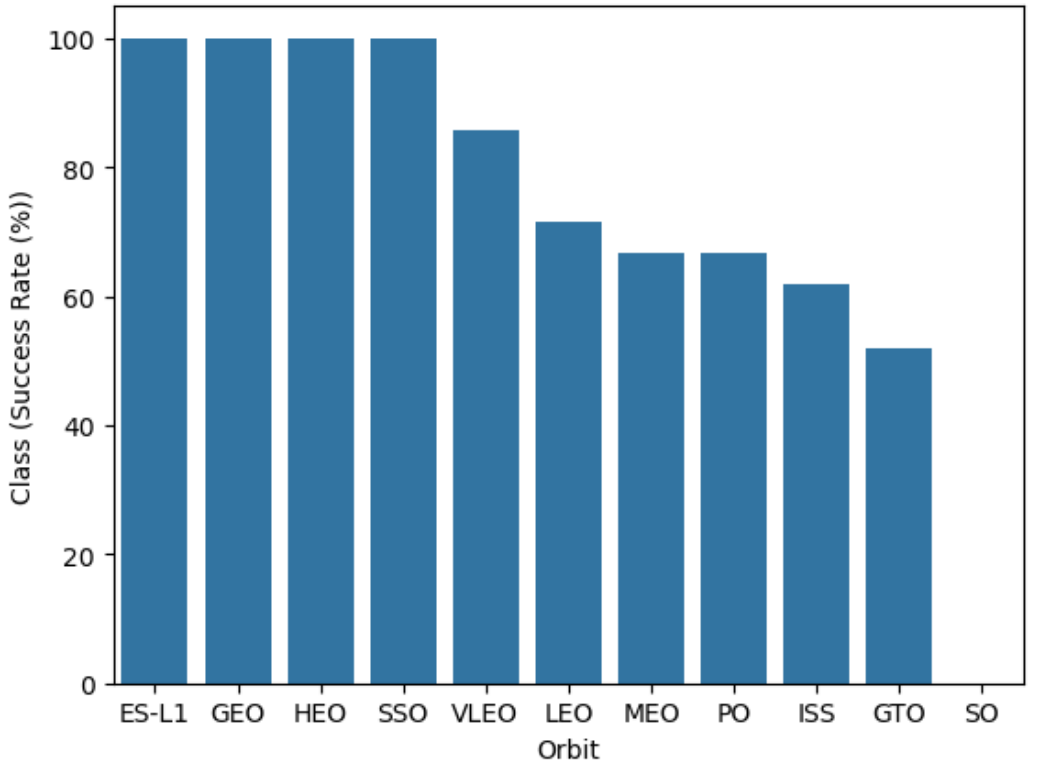


<u>Observation</u>-

- For the VAFB-SLC launchsite there are no rockets launched for heavypayload mass (greater than 10000).

Success Rate vs. Orbit Type

Success rate of each orbit

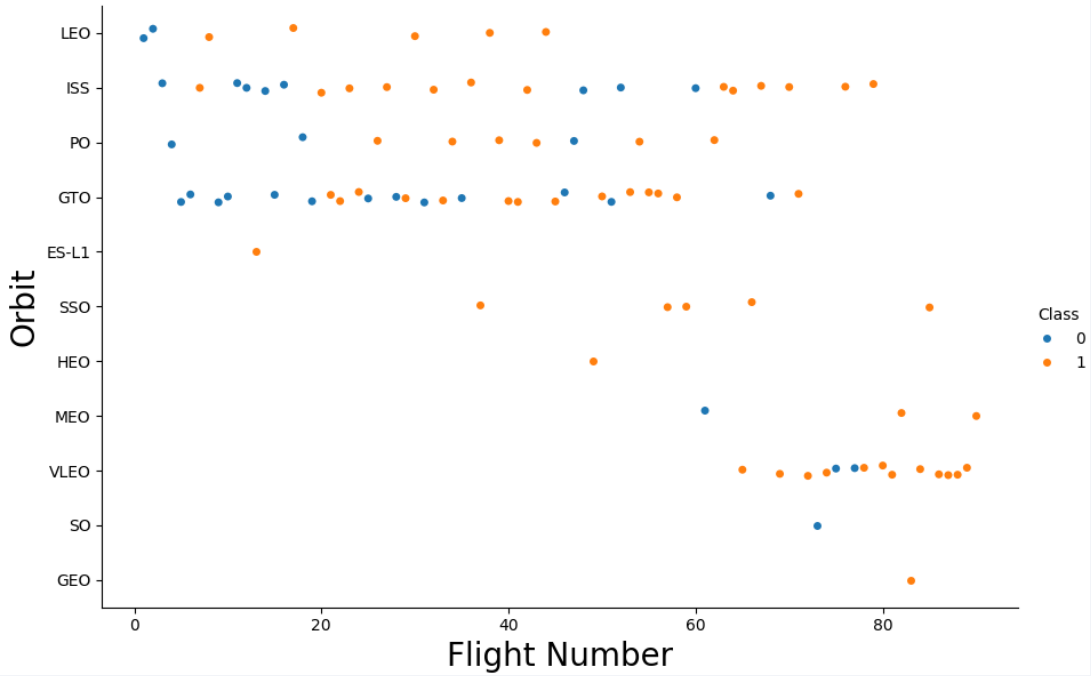


<u>Observation</u>-

- Orbits ES-L1, GEO, HEO & SSO have 100% success rates.
- GTO orbit having the lowest success rate at ~50%.
- SO orbit has 0% success rate.

Flight Number vs. Orbit Type

Flight Number vs Orbit

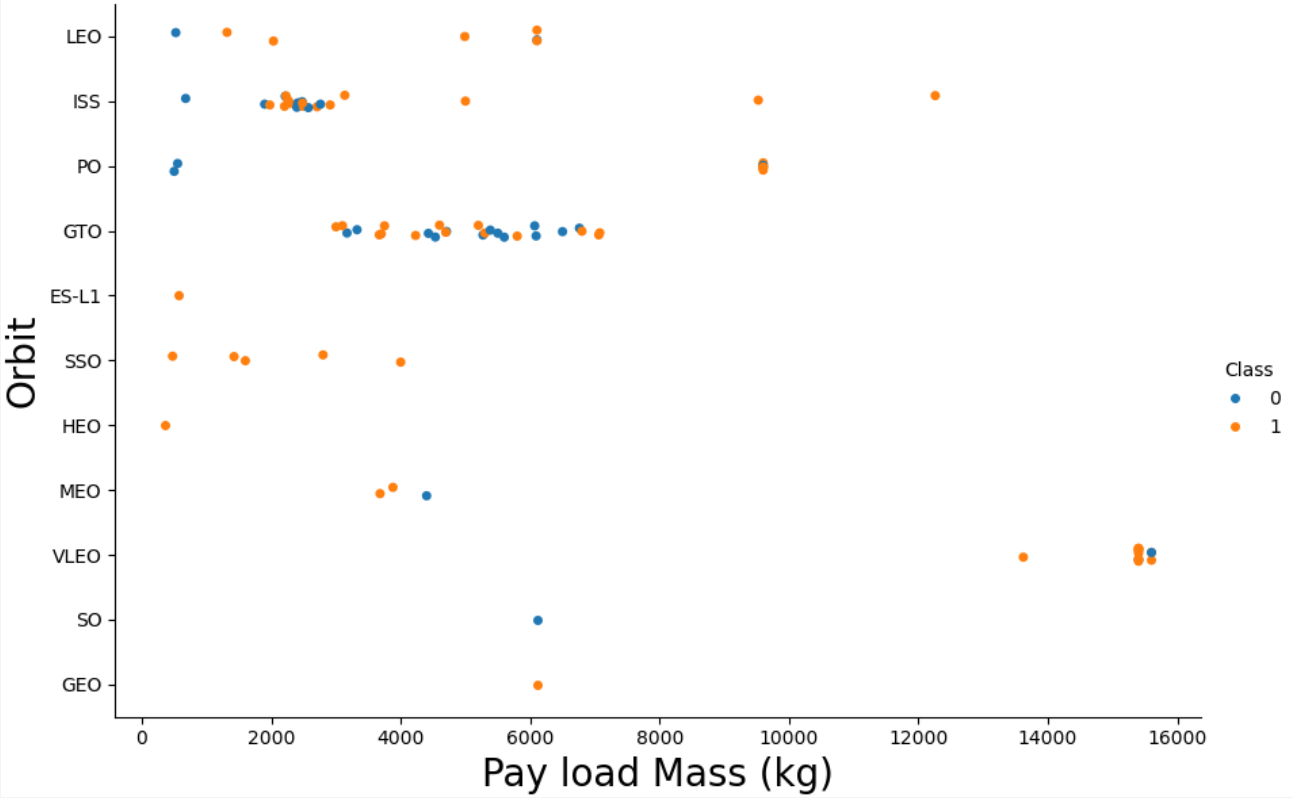


<u>Observation</u>-

- In the LEO orbit, as the number of flights increases from 10 the success rate also increases.
- Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.

Payload vs. Orbit Type

Pay Load Mass vs Orbit

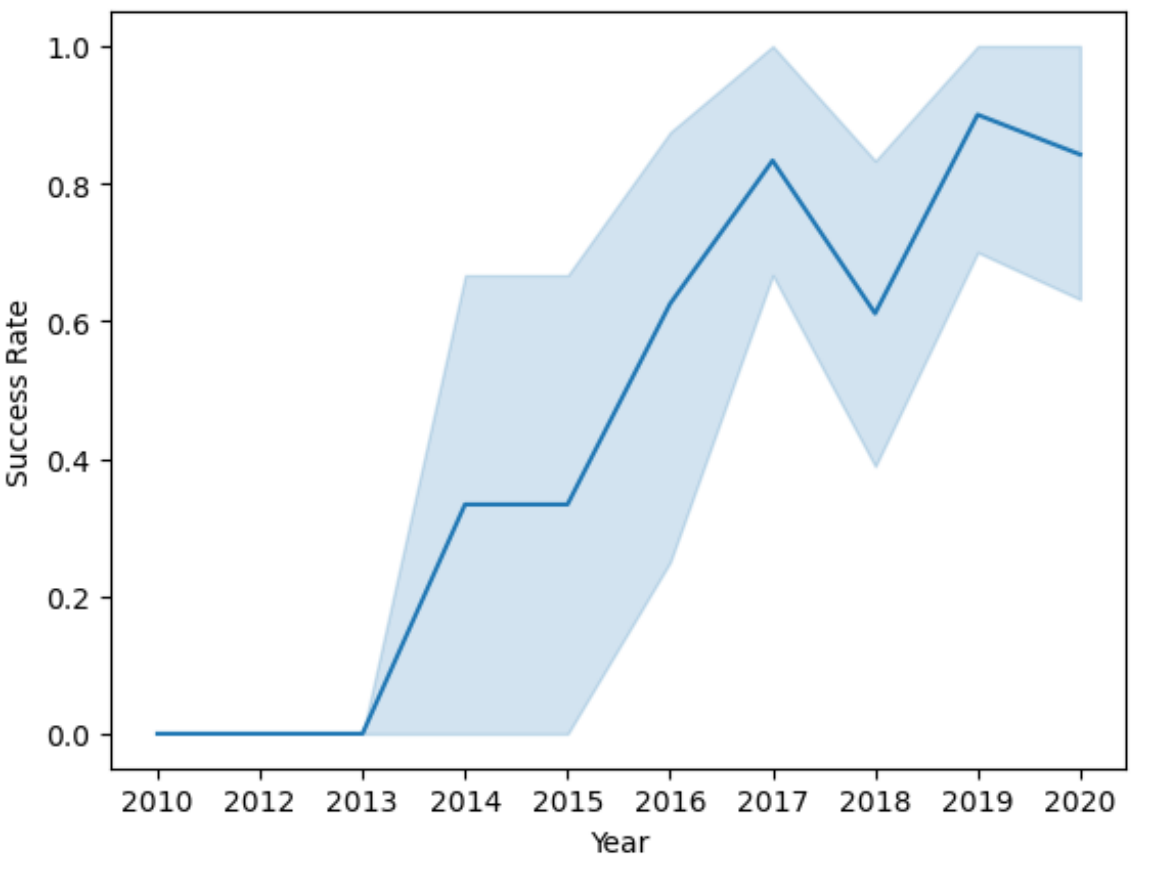


Observation-

- With heavy payloads the successful landing rates are more for Polar, LEO and ISS.
- For GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Launch Success Yearly Trend

Launch Success Yearly Trend



<u>Observation</u>-

- The success rate since 2013 kept increasing till 2020 with a small drop in 2018.

All Launch Site Names

```
In [20]: %sql SELECT DISTINCT LAUNCH_SITE as "Launch_Sites" FROM SPACEXTBL;

 * sqlite:///my_data1.db
Done.
```

Out[20]:

Launch_Sites
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

- Used the SQL ‘DISTINCT’ query on table ‘SPACEXTBL’ to find all the launch site names.

Launch Site Names Begin with 'CCA'

In [21]:
```
%sql SELECT * FROM 'SPACEXTBL' WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[21]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- Used SQL ‘WHERE’ and ‘LIKE’ query and wild card ‘CCA%’ to filter launch site names ‘FROM’ table ‘SPACEXTBL’.

Total Payload Mass

```
In [22]: %sql SELECT SUM(PAYLOAD_MASS__KG_) as "Total Payload Mass(Kgs)", Customer FROM 'SPACEXTBL' WHERE Customer = 'NASA (CRS)';

 * sqlite:///my_data1.db
Done.
```

Out[22]:

Total Payload Mass(Kgs)	Customer
45596	NASA (CRS)

- Used SUM() function to calculate total mass from ‘PAYLOAD_MASS__KG_’ column from customer ‘NASA (CRS)’.

Average Payload Mass by F9 v1.1

In [23]:
```
%sql SELECT AVG(PAYLOAD_MASS__KG_) as "Payload Mass Kgs", Customer, Booster_Version FROM 'SPACEXTBL' WHERE Booster_Version LIKE 'F9 v1.1%';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[23]:

Payload Mass Kgs	Customer	Booster_Version
2534.6666666666665	MDA	F9 v1.1 B1003

- Used AVG() function on ‘PAYLOAD_MASS__KG_’ column only for booster version ‘F9 v1.1%’.

First Successful Ground Landing Date

```
In [30]: %sql SELECT MIN(DATE) FROM 'SPACEXTBL' WHERE "Landing_Outcome" = "Success (ground pad)";

 * sqlite:///my_data1.db
Done.
```

Out[30]:

MIN(DATE)
2015-12-22

- Used ‘WHERE’ to filter the successful grounding and MIN() function on ‘DATE’ column to find the first successful landing date.

Successful Drone Ship Landing with Payload between 4000 and 6000

In [31]:
```
%sql SELECT DISTINCT Booster_Version, Payload FROM SPACEXTBL WHERE "Landing_Outcome" = "Success (drone ship)" AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[31]:

Booster_Version	Payload
F9 FT B1022	JCSAT-14
F9 FT B1026	JCSAT-16
F9 FT B1021.2	SES-10
F9 FT B1031.2	SES-11 / EchoStar 105

- Used condition ‘PAYLOAD__MASSKG_ > 4000 AND ‘PAYLOAD__MASSKG_ < 6000’ to filter the successful landing payload (WHERE ‘Landing_Outcome’).
- Used ‘DISTINCT’ query on Booster_Version and Payload to find the successful drone ship landing payload.

Total Number of Successful and Failure Mission Outcomes

In [26]:
```
%sql SELECT "Mission_Outcome", COUNT("Mission_Outcome") as Total FROM SPACEXTBL GROUP BY "Mission_Outcome";
```

```
 * sqlite:///my_data1.db
Done.
```

Out[26]:

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

- Used GROUP BY query to group ‘Mission_Outcome’ column.
- Used COUNT() method to sum the grouped outcome to find the total number of successful and failed missions.

Boosters Carried Maximum Payload

In [27]:
```
%sql SELECT "Booster_Version",Payload, "PAYLOAD_MASS__KG_" FROM SPACEXTBL WHERE "PAYLOAD_MASS__KG_" = (SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTBL);
```

```
 * sqlite:///my_data1.db
Done.
```

Out[27]:

Booster_Version	Payload	PAYLOAD_MASS__KG_
F9 B5 B1048.4	Starlink 1 v1.0, SpaceX CRS-19	15600
F9 B5 B1049.4	Starlink 2 v1.0, Crew Dragon in-flight abort test	15600
F9 B5 B1051.3	Starlink 3 v1.0, Starlink 4 v1.0	15600
F9 B5 B1056.4	Starlink 4 v1.0, SpaceX CRS-20	15600
F9 B5 B1048.5	Starlink 5 v1.0, Starlink 6 v1.0	15600
F9 B5 B1051.4	Starlink 6 v1.0, Crew Dragon Demo-2	15600
F9 B5 B1049.5	Starlink 7 v1.0, Starlink 8 v1.0	15600
F9 B5 B1060.2	Starlink 11 v1.0, Starlink 12 v1.0	15600
F9 B5 B1058.3	Starlink 12 v1.0, Starlink 13 v1.0	15600
F9 B5 B1051.6	Starlink 13 v1.0, Starlink 14 v1.0	15600
F9 B5 B1060.3	Starlink 14 v1.0, GPS III-04	15600
F9 B5 B1049.7	Starlink 15 v1.0, SpaceX CRS-21	15600

- Used WHERE condition with subquery to SELECT MAX payload.

2015 Launch Records

```
In [40]: %sql SELECT substr(Date,0,5), substr(Date, 6, 2),"Booster_Version", "Launch_Site", Payload, "PAYLOAD_MASS__KG_", "Mission_Outcome", "Landing_Outcome" FROM SPACEXTBL WHERE substr(Date,0,5)='2015' AND "Landing_Outcome" = 'Failure (drone ship)';

 * sqlite:///my_data1.db
Done.
```

Out[40]:

substr(Date,0,5)	substr(Date, 6, 2)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Mission_Outcome	Landing_Outcome
2015	01	F9 v1.1 B1012	CCAFS LC-40	SpaceX CRS-5	2395	Success	Failure (drone ship)
2015	04	F9 v1.1 B1015	CCAFS LC-40	SpaceX CRS-6	1898	Success	Failure (drone ship)

- Used the subsrt() function to get the month and year from the ‘Date’ column.
- substr(Date,0,5)='2015' and Landing_outcome = ‘Failure (drone ship’) to filter the 2015 failed launch records.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

In [42]:
```
%sql SELECT * FROM SPACEXTBL WHERE "Landing_Outcome" LIKE 'Success%' AND (Date BETWEEN '2010-06-04' AND '2017-03-20') ORDER BY Date DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[42]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2017-02-19	14:39:00	F9 FT B1031.1	KSC LC-39A	SpaceX CRS-10	2490	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
2017-01-14	17:54:00	F9 FT B1029.1	VAFB SLC-4E	Iridium NEXT 1	9600	Polar LEO	Iridium Communications	Success	Success (drone ship)
2016-08-14	5:26:00	F9 FT B1026	CCAFS LC-40	JCSAT-16	4600	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2016-07-18	4:45:00	F9 FT B1025.1	CCAFS LC-40	SpaceX CRS-9	2257	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
2016-05-27	21:39:00	F9 FT B1023.1	CCAFS LC-40	Thaicom 8	3100	GTO	Thaicom	Success	Success (drone ship)
2016-05-06	5:21:00	F9 FT B1022	CCAFS LC-40	JCSAT-14	4696	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
2016-04-08	20:43:00	F9 FT B1021.1	CCAFS LC-40	SpaceX CRS-8	3136	LEO (ISS)	NASA (CRS)	Success	Success (drone ship)
2015-12-22	1:29:00	F9 FT B1019	CCAFS LC-40	OG2 Mission 2 11 Orbcomm-OG2 satellites	2034	LEO	Orbcomm	Success	Success (ground pad)

- Used condition LIKE query for ‘Success%’ AND Date filter.
- ORDER date by descending.
- This gives the rank landing outcome between 2010-06-04 and 2017-03-20.

Section 3

Launch Sites Proximities Analysis

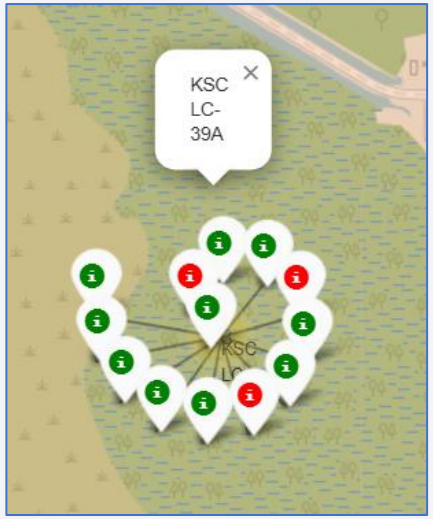
All Launch Site Locations

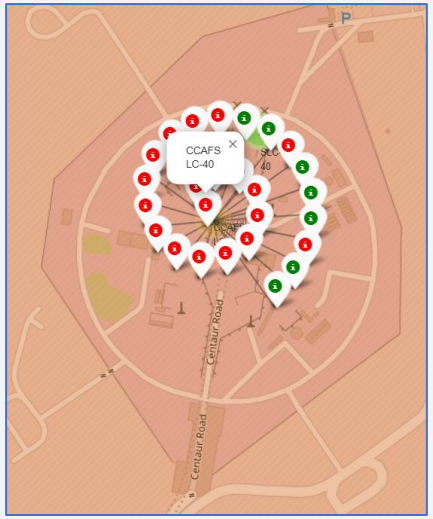
	Launch Site	Lat	Long
0	CCAFS LC-40	28.562302	-80.577356
1	CCAFS SLC-40	28.563197	-80.576820
2	KSC LC-39A	28.573255	-80.646895
3	VAFB SLC-4E	34.632834	-120.610745

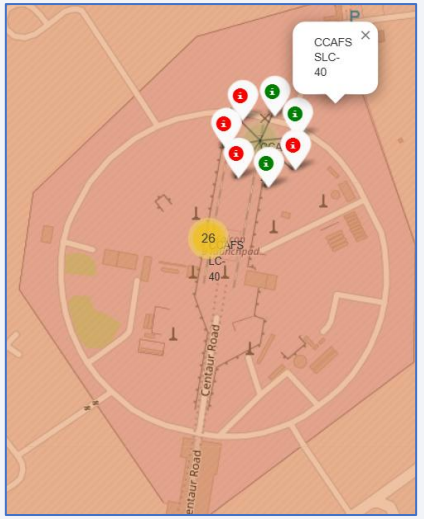
- All launch sites are near the equator but not that close to the equator.
- All launch sites are very close to coast.

Launch Site Outcome

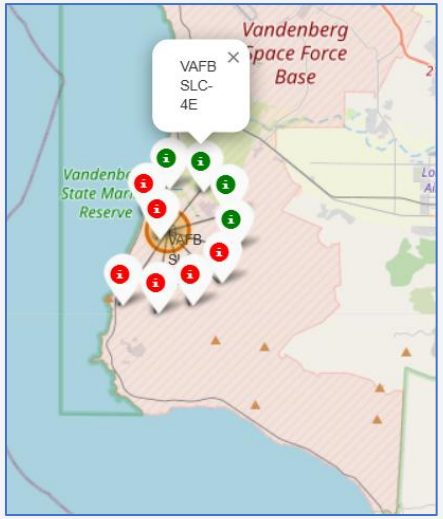
EAST COAST FLORIDA







WEST COAST CALIFORNIA



- East coast launch site KSC LC-39A (77%) has relatively high success rate compared to CCAFS LC-40 (26%) & CCAFS SLC-40 (43%).
- West coast launch site VAFB SLC-4E (40%) has less success rate compared to sites KSC LC-39A and CCAFS SLC-40.

Launch Site to Proximities Distances

Launch site CCAFS SLC-40 to highway (Washington Avenue) is 23.19km

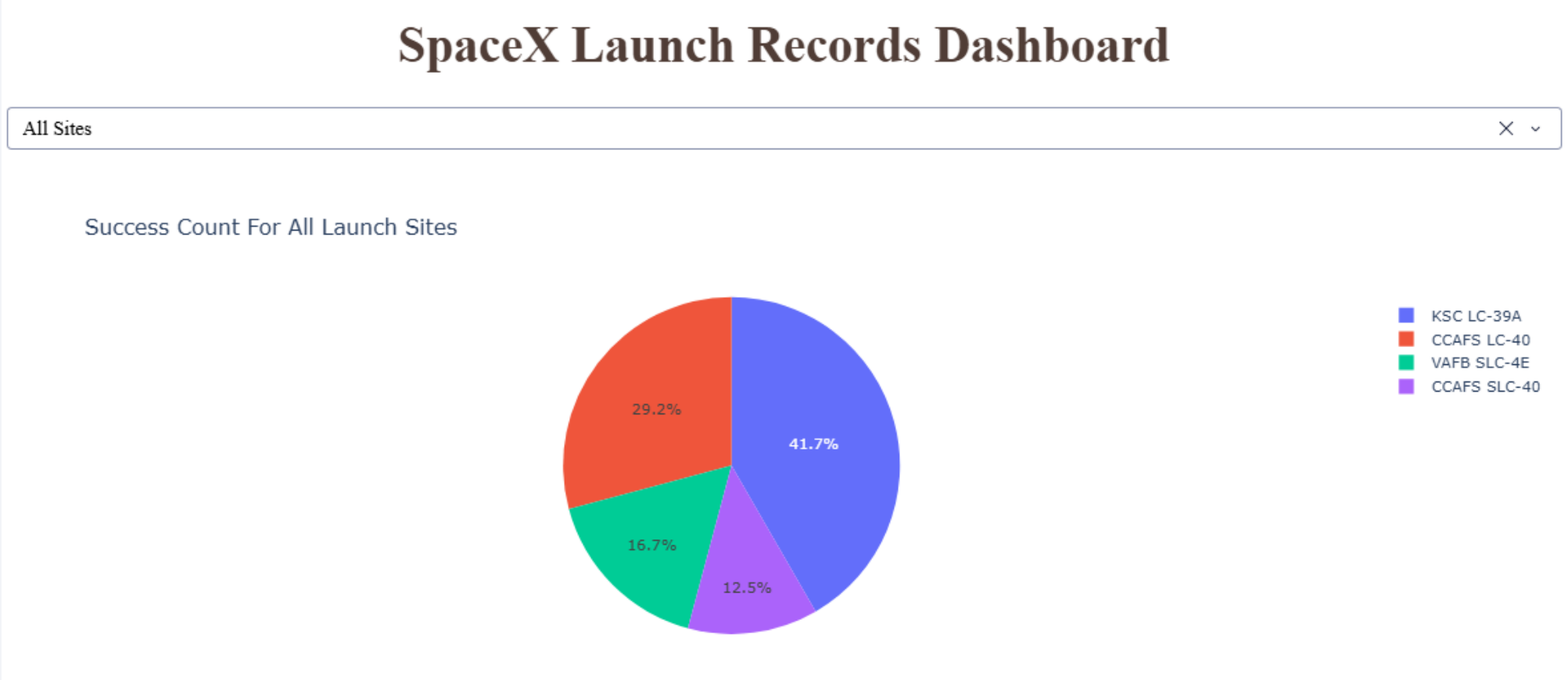
Launch site CCAFS SLC-40 to coastline is 0.86km.

- Launch sites are in close proximity to railways, highways and coastlines.
- Launch sites are away from cities.

Section 4

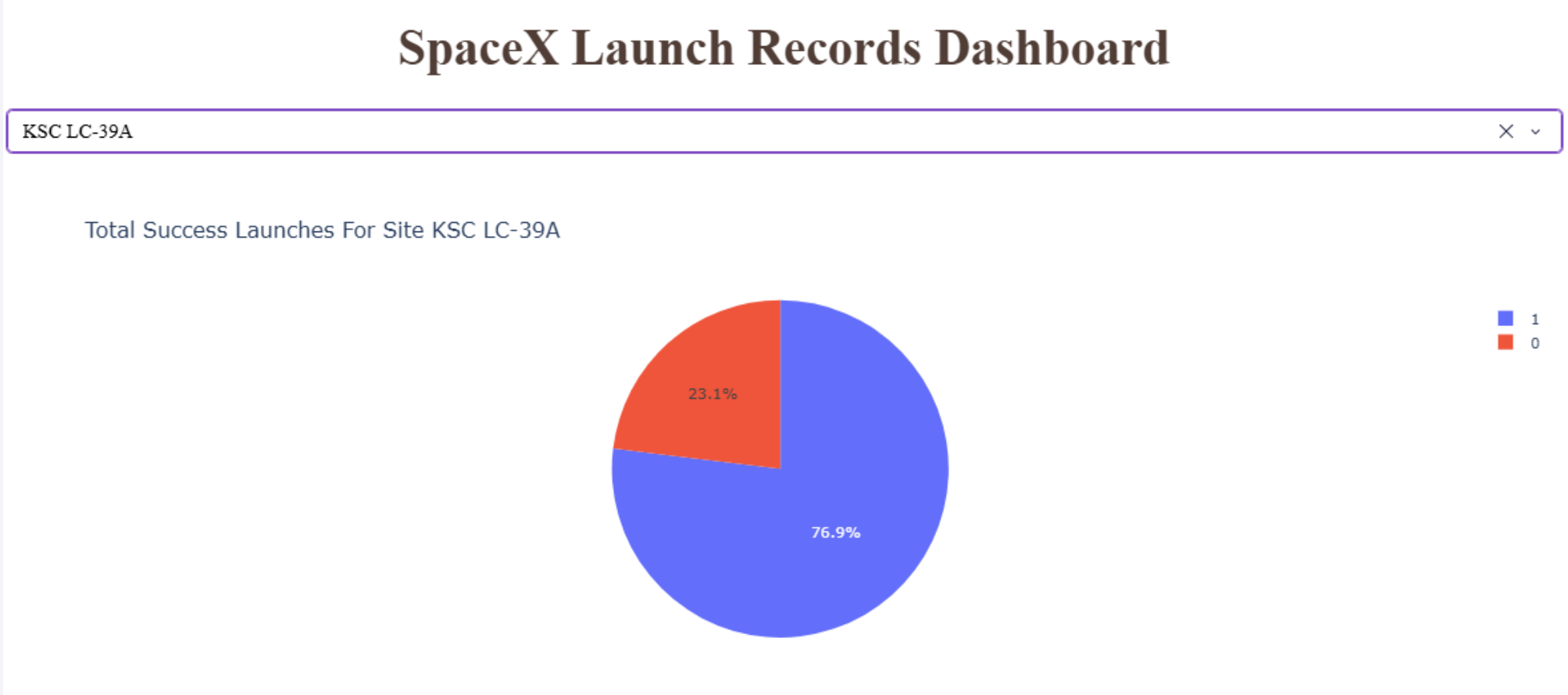
Build a Dashboard with Plotly Dash

All Sites Success Launch Record



- Launch site KSC LC-39A has the highest launch success rate at 42% followed by CCAFS LC-40 at 29%, VAFB SLC-4E at 17% and CCAFS SLC-40 at 13%.

Highest Launch Success Site



- Launch site KSC LC-39A has the highest success ratio of 77% against 23% failed launches.

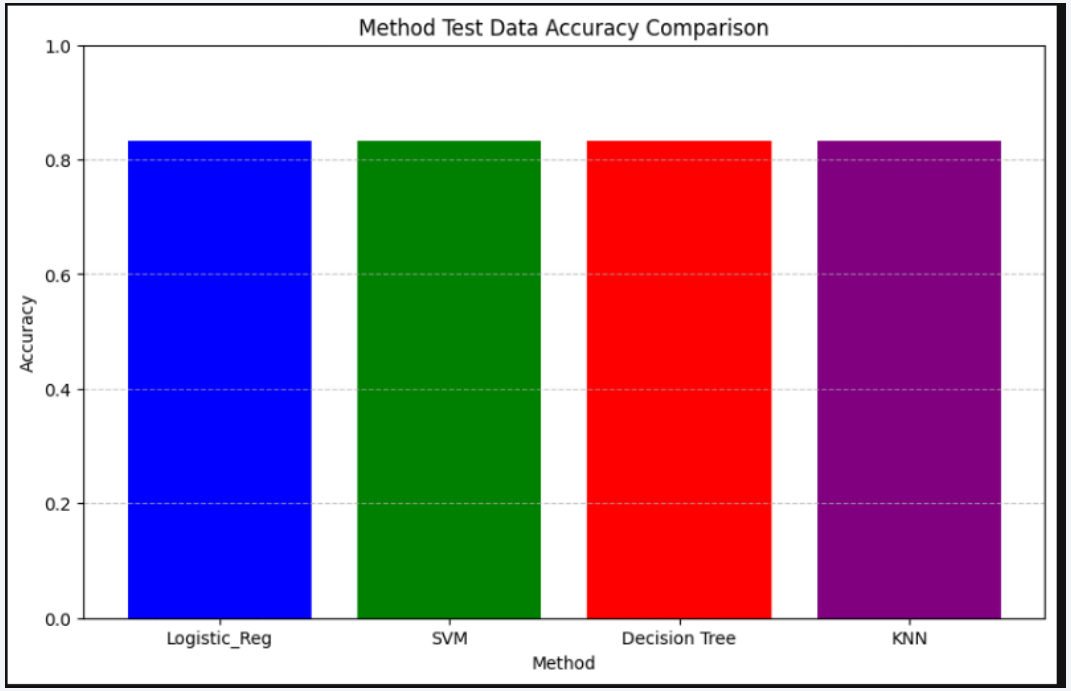
Payload vs Launch Outcome

- KSC LC-39A site has the largest successful launches of 41.7%. of the total successful launches.
- KSC LC-39A site has the highest launch success rate of 76.9%.
- The highest launch success rate has a payload between 3,000 Kg and 4,000 Kg with 7 launches.
- The lowest launch success rate has a payload between 6,000 Kg and 9,000 Kg with 1 launch.
- The FT Booster version has the highest launch success rate of 13 launches.

Section 5

Predictive Analysis (Classification)

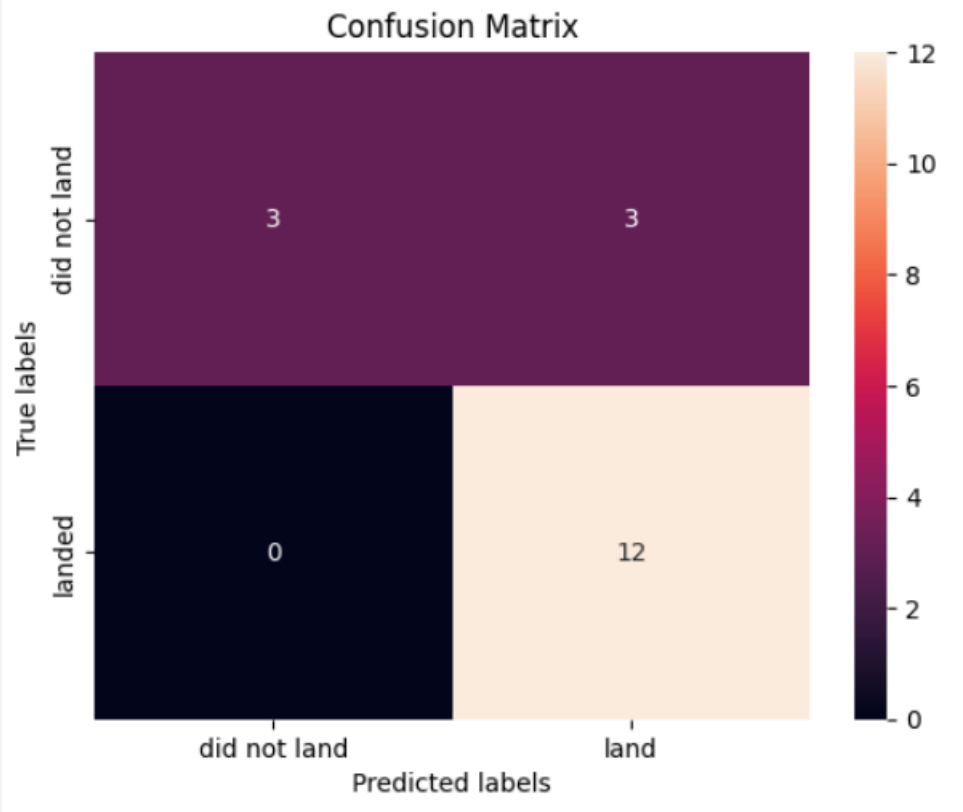
Classification Accuracy



Method	Test Data Accuracy
Logistic_Reg	0.833333
SVM	0.833333
Decision Tree	0.833333
KNN	0.833333

- For all the methods like Logistic Regression, SVM, Decision Tree and KNN the data accuracy score is same 83.33%.
- So, all methods perform equally.

Confusion Matrix



- All the models have same confusion matrix.
- So, all methods perform equally.

Conclusions

- As the flight number increases, the first stage is more likely to land successfully. E.g.:-
  - After flight no. 50, the success rate for the VAFB SLC 4E launch site is 100%
  - After flight no. 80, both KSC LC 39A and CCAFS SLC 40 have a 100% success rates.
- With more massive payloads > 9000 kg, the first stage often returns successfully.
- With heavy payloads the successful landing rates are more for Polar, LEO and ISS orbits.
- The highest launch success rate has a payload between 3,000 Kg and 4,000 Kg with 7 launches.
- Orbits ES-L1, GEO, HEO & SSO have 100% success rates.
- In the LEO orbit, success rate increases for no. of flights >10.
- All launch sites are close to the proximities of railways, highways and coastlines but also away from cities.
- Launch site KSC LC-39A has the highest launch success.
- The success rate since 2013 kept increasing till 2020.

So, in order for a new company to bid against SpaceX all the above parameters should be considered for a successful return of first stage.

Appendix

Below are series of helper functions that were used for data manipulation-

- From the API extract information using identification numbers in the launch data (e.g. -rocket column).

```python
# Takes the dataset and uses the rocket column to call the API and append the data to the list
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
            BoosterVersion.append(response['name'])
```

- Process web scraped HTML table.

```python
def date_time(table_cells):
    """
    This function returns the data and time from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
    This function returns the booster version from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=''.join([booster_version for i,booster_version in enumerate( table_cells.strings) if i%2==0][0:-1])
    return out
```

- Function to plot the confusion matrix.

```python
def plot_confusion_matrix(y,y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax= plt.subplot()
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix');
    ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()
```

Thank you!